



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 32

Dr. Asad Mahmood

April 20, 2026

Lecture Outline

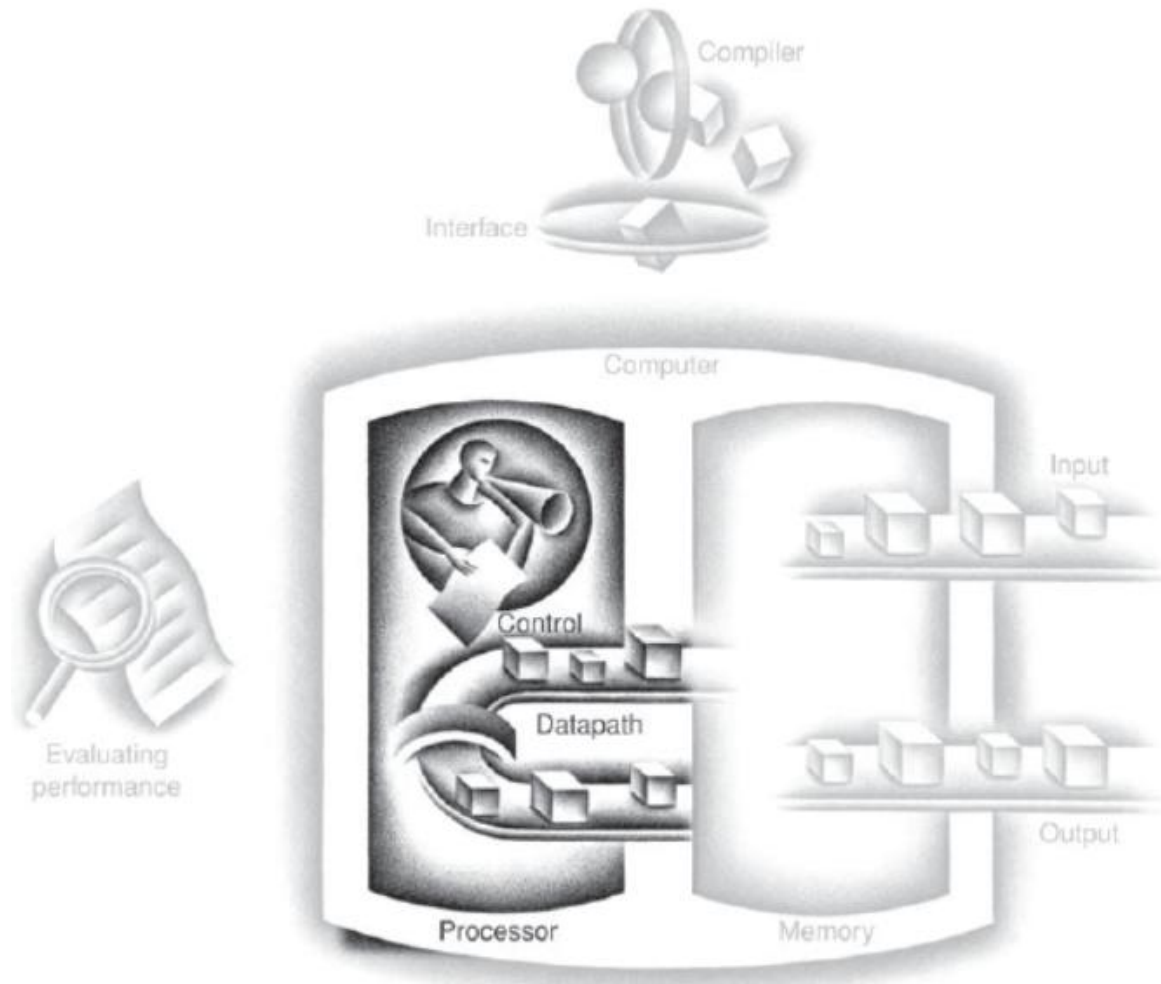
- Announcements (Assignment and Quiz 4, Prof. Qadir talk, Wed Class)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.6 An Overview of Pipelining
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.6 An Overview of Pipelining
 - 4.7 Pipelined Datapath and Control

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



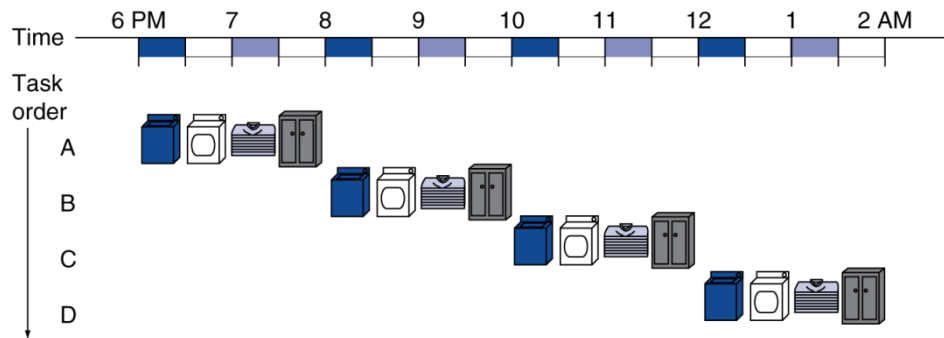
Chapter Outline

Chapter 4 – The Processor

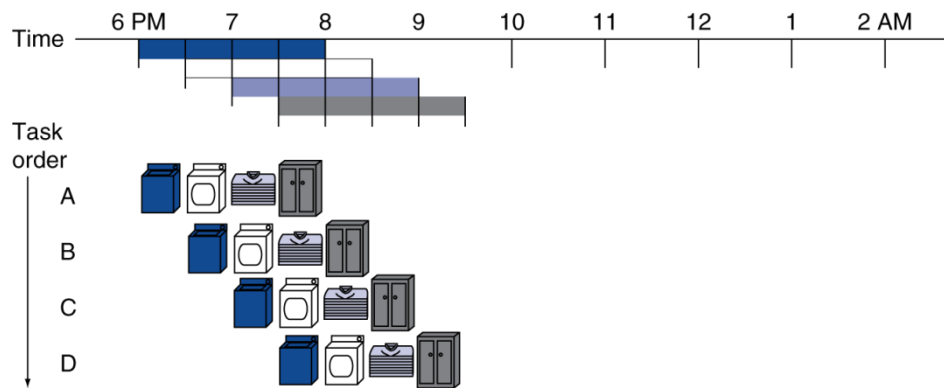
- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Pipelining Analogy

- **Pipelined laundry**: overlapping execution
 - **Parallelism** improves performance



- **Four loads:**
 - **Speedup?**



- **Twenty loads:**
 - **Speedup?**

RISC-V Pipeline

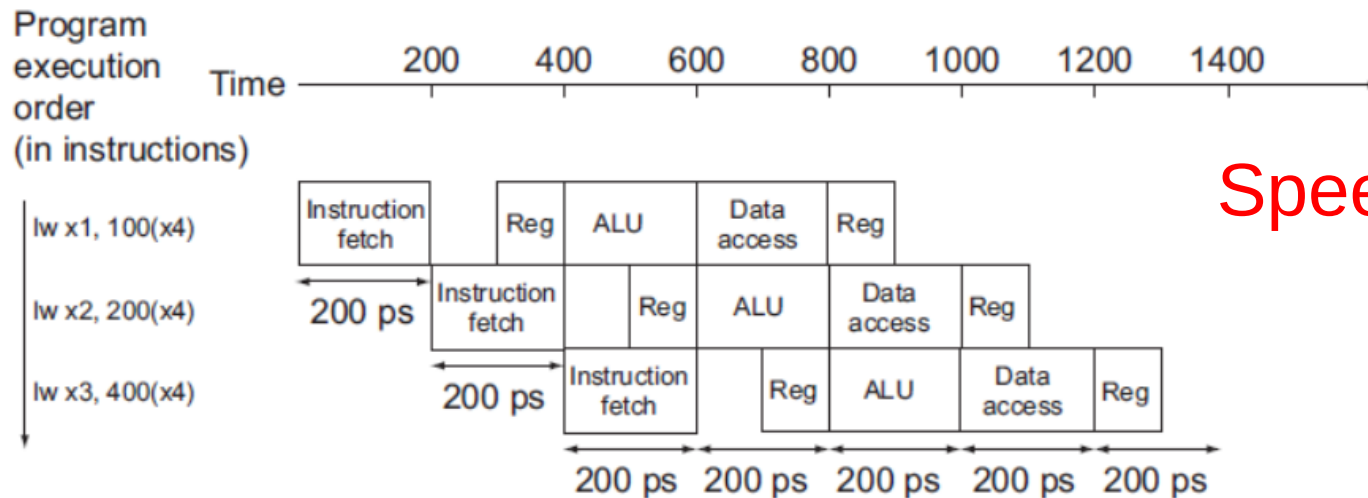
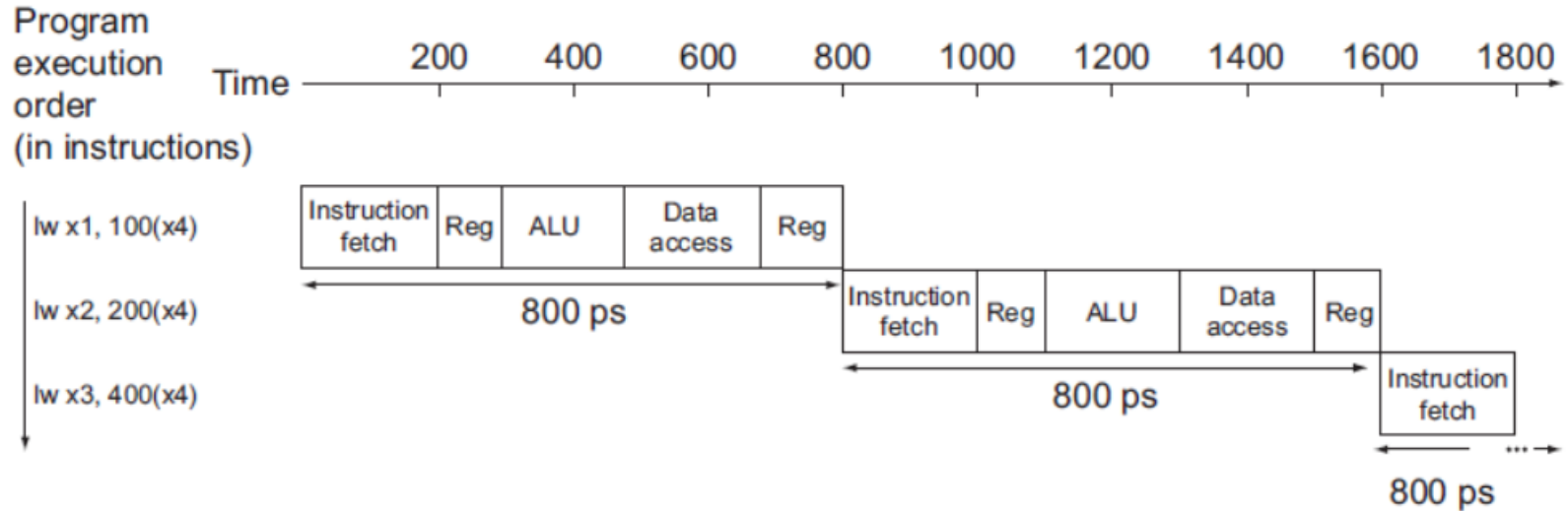
- Five stages, one step per stage
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register read
 3. EX: Execute operation or calculate address
 4. MEM: Access memory operand (if necessary)
 5. WB: Write result back to register (if necessary)

Pipeline Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath (lw, sw, R-format, beq)
- Assumptions made?

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance



Speedup?

Pipeline Speedup

- Time between instructions assuming all stages are balanced
- If not balanced, speedup is less
- Pipelining improves performance by *increasing instruction throughput, in contrast to decreasing the execution time of an instruction*

Pipelining and ISA Design

- RISC-V ISA designed for pipelining
 - All instructions are 32-bits
 - Easier to fetch and decode in one cycle
 - c.f. x86: 1- to 17-byte instructions
 - Few and regular instruction formats
 - Can decode and read registers in one step
 - ALU does not need memory operands
 - Can calculate address in 3rd stage, access memory in 4th stage
 - Saves us from a long pipeline!

Hazards

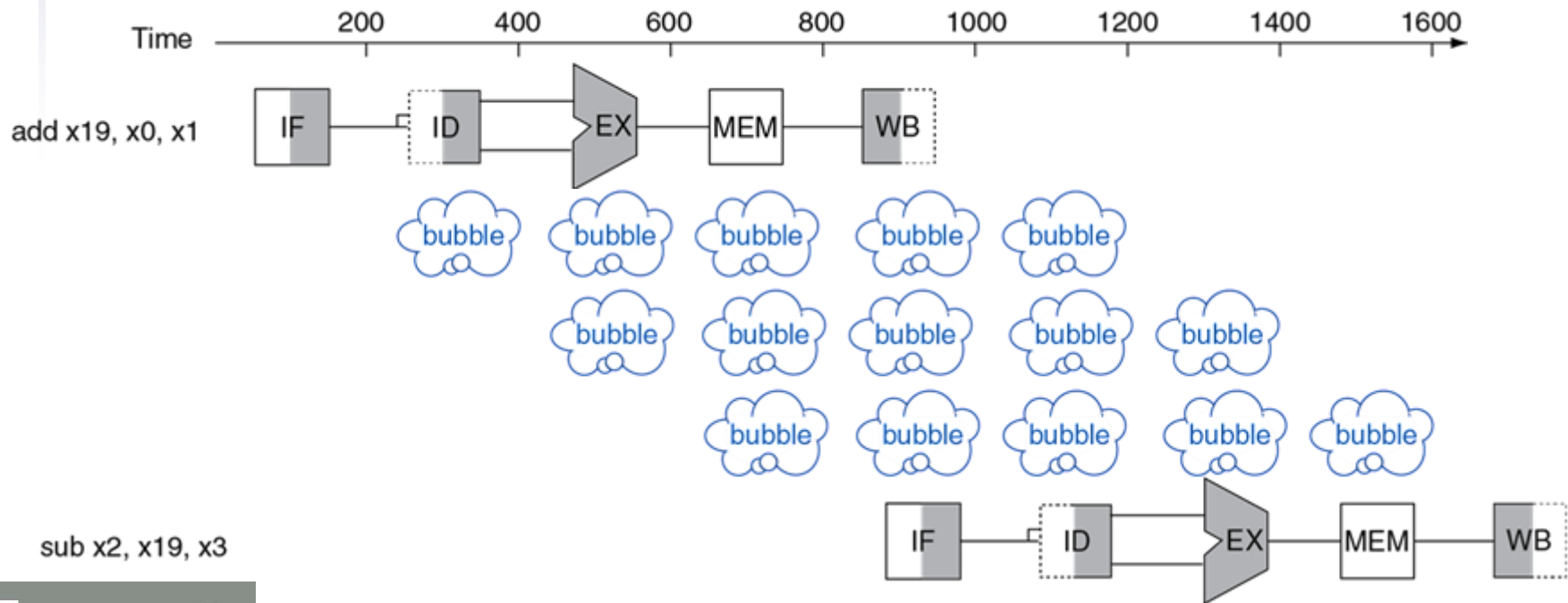
- **Hazards** are situations that prevent starting the next instruction in the next cycle
- Three types:
 1. Structural hazards
 2. Data hazards
 3. Control hazards

Structural Hazards

- **Conflict** for use of a resource (laundry analogy)
- In RISC-V pipeline with a single memory
 - Load/store requires data access
 - **Instruction fetch would have to *stall*** for that cycle – see previous example
 - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories/caches

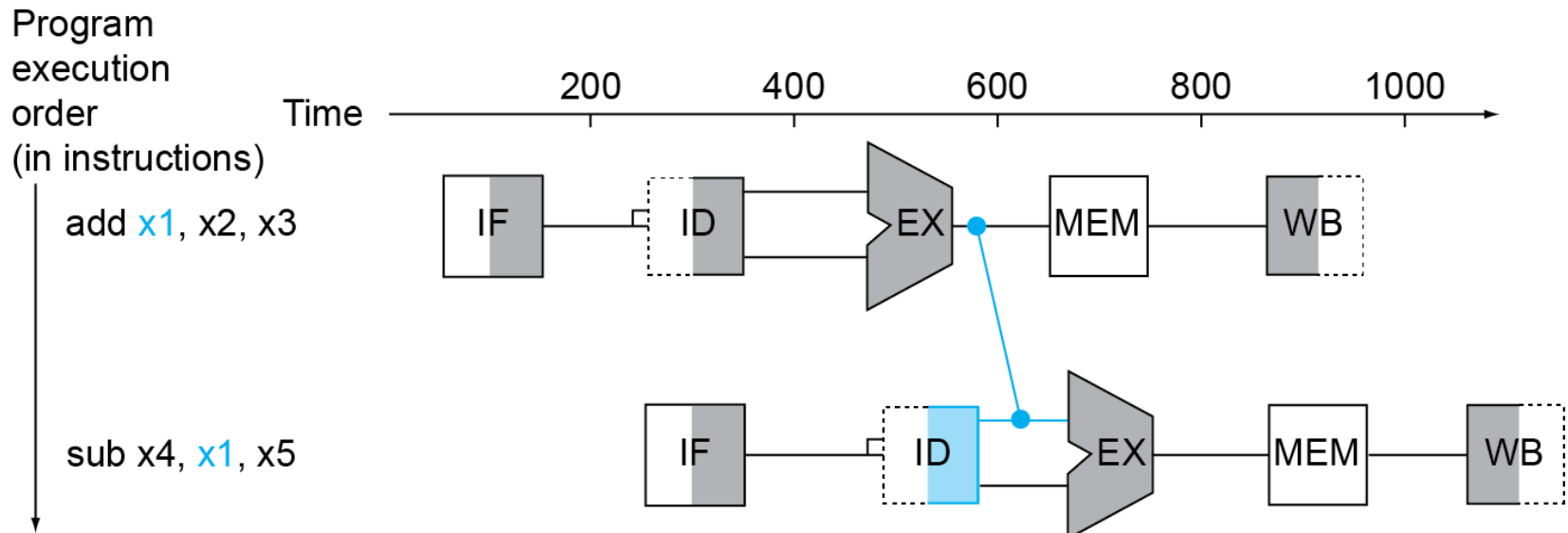
Data Hazards

- An instruction depends on completion of data access by a previous instruction
 - add x19, x0, x1
 - sub x2, x19, x3



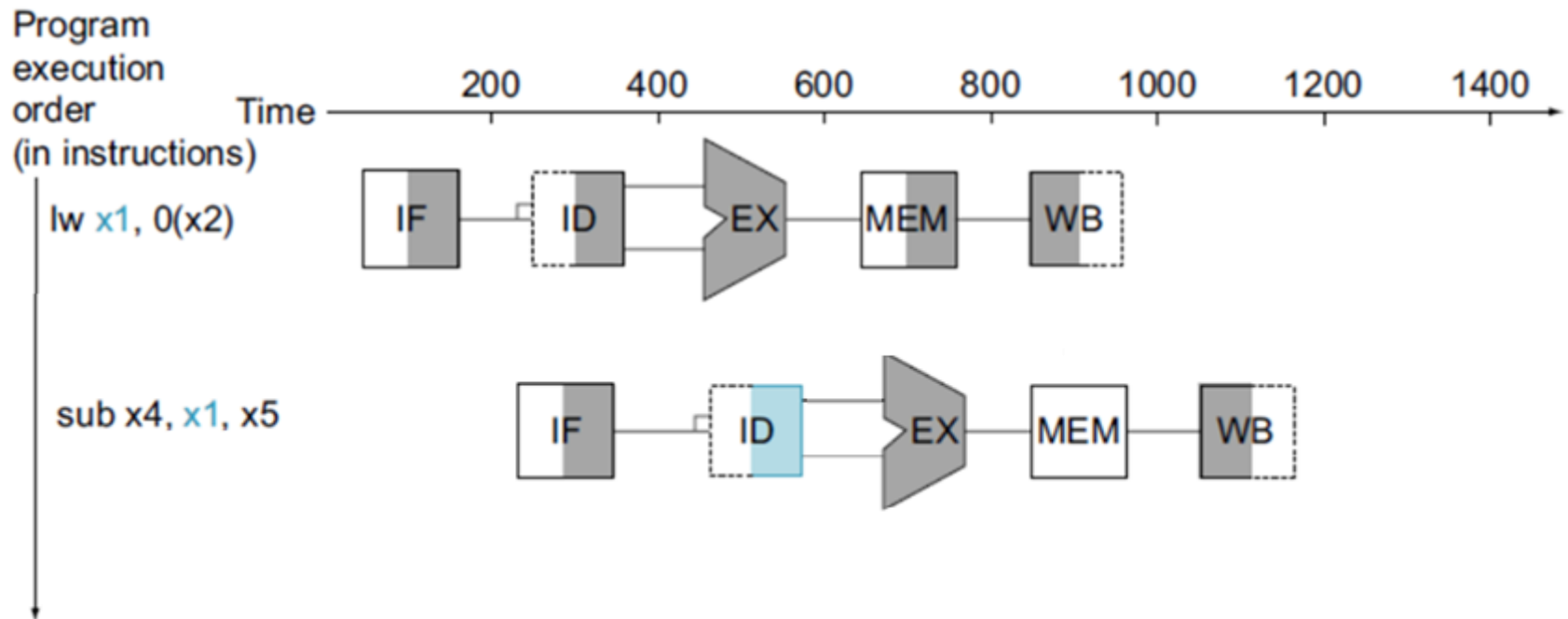
Forwarding (aka Bypassing)

- Use result as soon as it is computed
 - Requires extra connections in the datapath



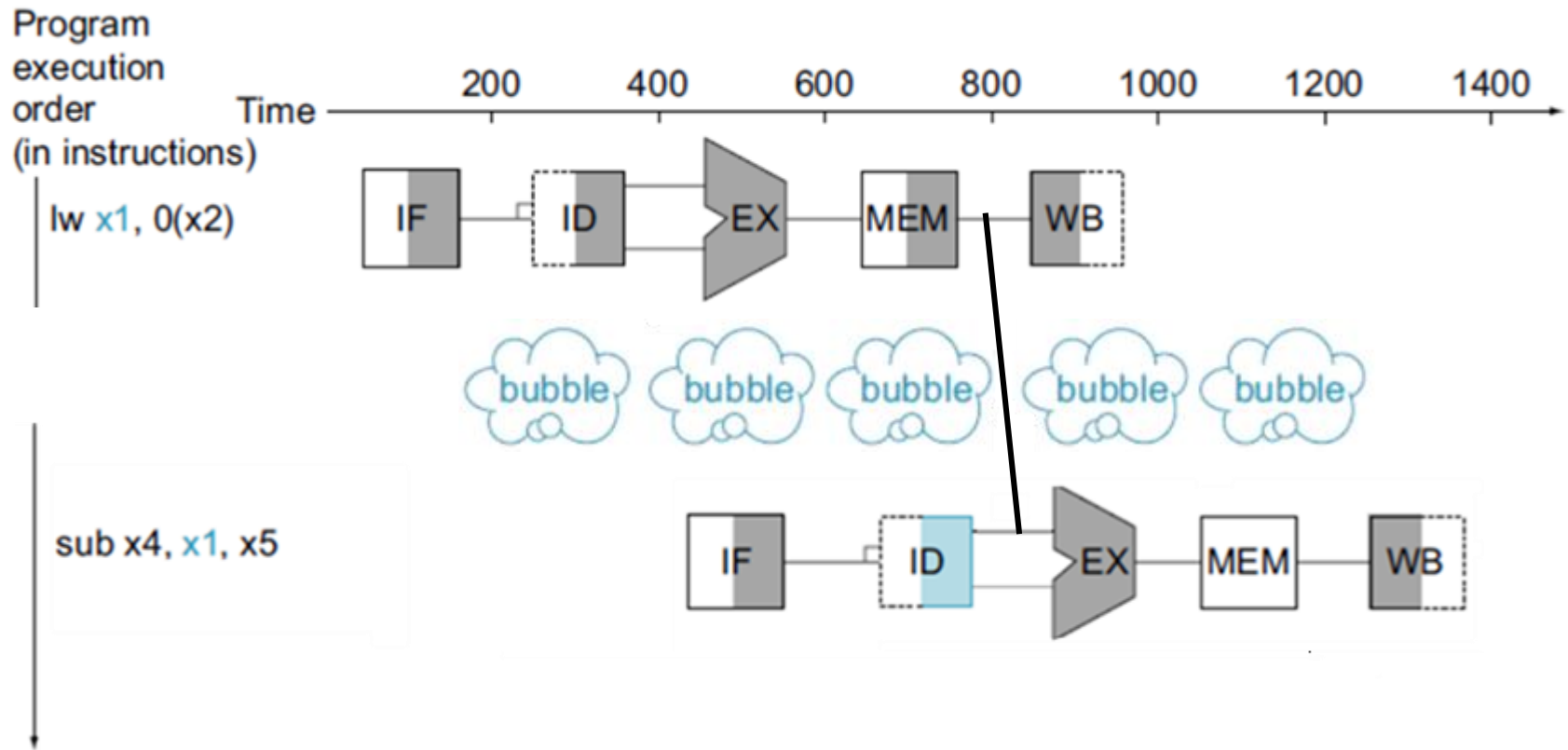
Load-Use Data Hazard

- Can't always avoid stalls by forwarding
 - If value not computed when needed
 - Need to stall! (Add a bubble)
 - Solution also possible via software re-ordering



Load-Use Data Hazard

- Can't always avoid stalls by forwarding



Code Scheduling to Avoid Stalls

- Reorder code to avoid use of load result in the next instruction
- C code for $a = b + e; c = b + f;$
- Stalls?

<code>lw x1, 0(x31)</code>	<code>// Load b</code>
<code>lw x2, 4(x31)</code>	<code>// Load e</code>
<code>add x3, x1, x2</code>	<code>// b + e</code>
<code>sw x3, 12(x31)</code>	<code>// Store a</code>
<code>lw x4, 8(x31)</code>	<code>// Load f</code>
<code>add x5, x1, x4</code>	<code>// b + f</code>
<code>sw x5, 16(x31)</code>	<code>// Store c</code>

Total cycles ?

Code Scheduling to Avoid Stalls

- **Reorder code** to avoid use of load result in the next instruction
- C code for $a = b + e; c = b + f;$

```
lw    x1, 0(x31)
lw    x2, 4(x31)
lw    x4, 8(x31)
add   x3, x1, x2
sw    x3, 12(x31)
add   x5, x1, x4
sw    x5, 16(x31)
```

Total cycles

Control / Branch Hazards

- Branch determines flow of control
 - Fetching next instruction depends on branch outcome
 - Pipeline can't always know what the next instruction should be
- In RISC-V pipeline
 - Need to compare registers and compute target early in the pipeline
 - Add hardware to do it in ID stage

Stall on Branch

- Wait until branch outcome determined **before fetching next instruction**
 - Using **extra hardware**
- **Performance impact** (SPECint2006 with CPI = 1, 10% conditional branches, **impact?**)

